CSE470: Software Engineering

Software Metrics Part 1

What is a CFG

An abstract representation of a structured program, function or method.

Consists of two major components:

- **Node**: Represents a stretch of sequential code statements with no branches.
- **Directed Edge (also called arc):** Represents a branch, alternative path in execution.

Path: A collection of Nodes linked with Directed Edges.

Specifications of a CFG

A CFG should have:

- 1 entry arc (known as a directed edge, too).
- 1 exit arc.

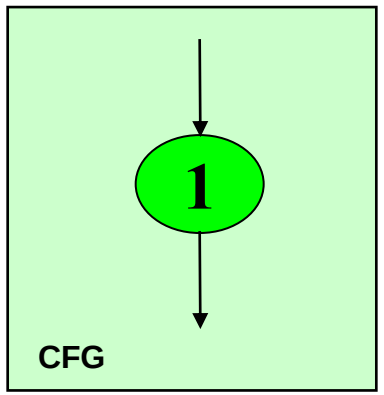
All nodes should have:

- At least 1 entry arc.
- At least 1 exit arc.

Statements

All general statements can be represented as one node as there is no branch.

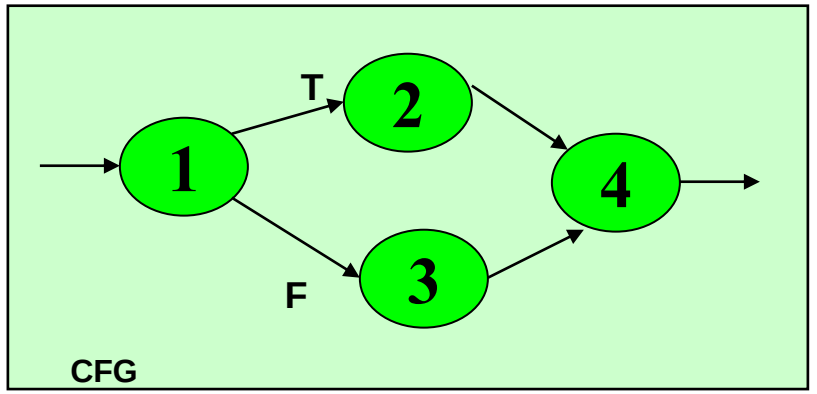
Statement 1;
Statement 2;
Statement 3;
Statement 4;



Conditions

All general statements can be represented as one node as there is no branch.

```
if X > 0: {1}
    Statement1; {2}
else:
    Statement2; {3}
```



{} -> is denoting the node numbers

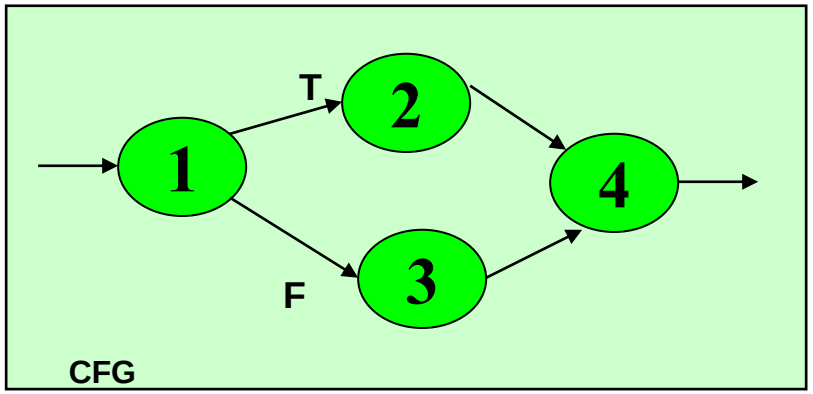
Logical Node

A Logical Node that does not represent any actual statements can be added as a joining point for several incoming edges.

Represents a logical closure.

Example:

Node 4 from previous example



Loops

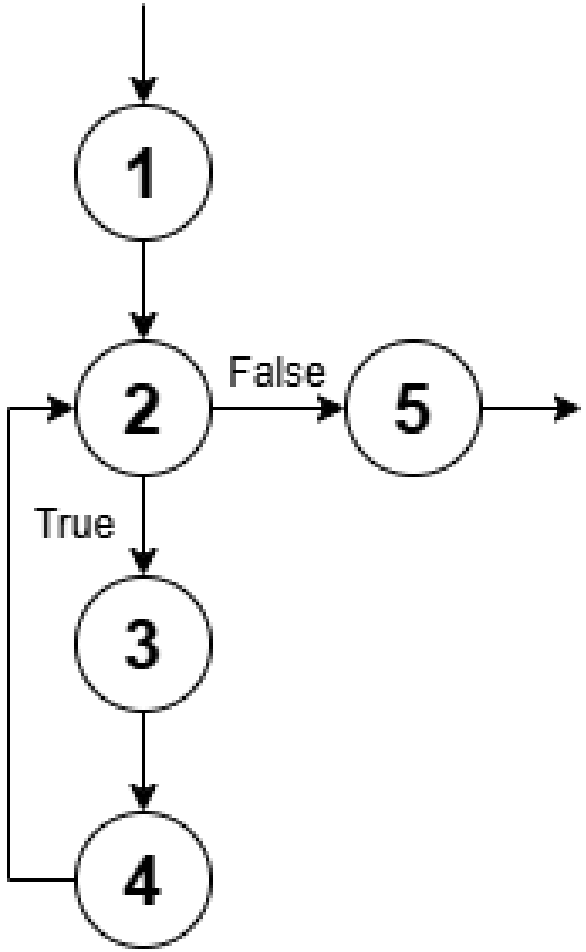
Every loop has 3 parts: Initialization, Condition, Increment or Decrement

So in CFG we identify each of them as a separate node.

Here we will discuss 2 types of loops: While and For

While Loops

```
n = 0 -> Initialization {1}
while (n < 5): -> Condition {2}
      print(n) -> Body Statement {3}
      n += 1 ->Increment {4}
```



Node-5 is a logical node here

For Loops

Python implement for-loop a bit differently than others. It achieves the same functionality as the other for-loops using the range() function.

range() has 3 main parameters: Start, Stop and Step

To simplify things, we will consider the following:

Start -> Initialization

Stop -> Condition

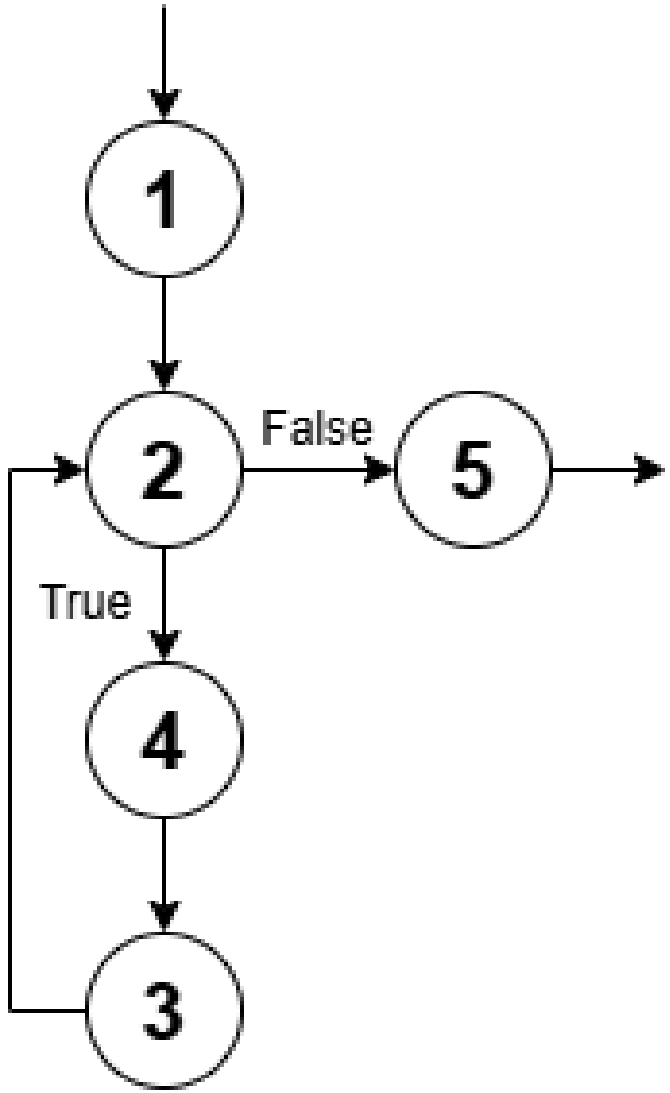
Step -> Increment/Decrement

For Loops

```
for i in range(0, 5, 1): #{1, 2, 3}
    print(n) #{4}

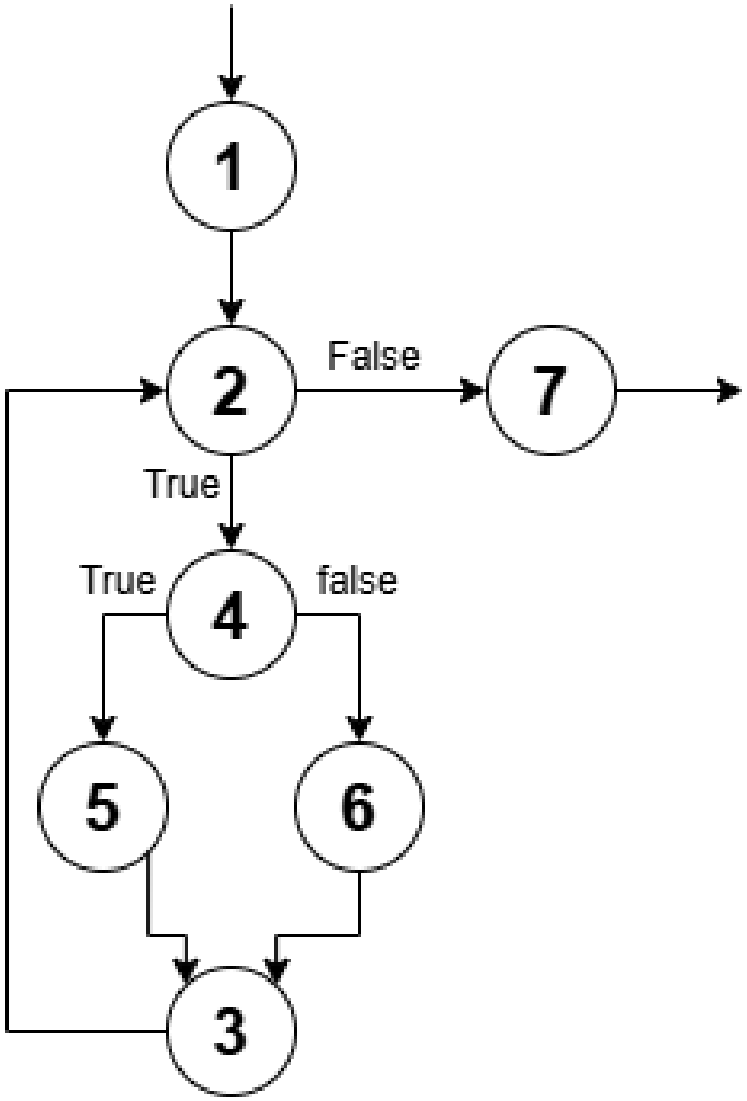
for i in range(0, 5): #{1, 2, 3}
    print(n) #{4}

for i in range(5): #{1, 2, 3}
    print(n) #{4}
```



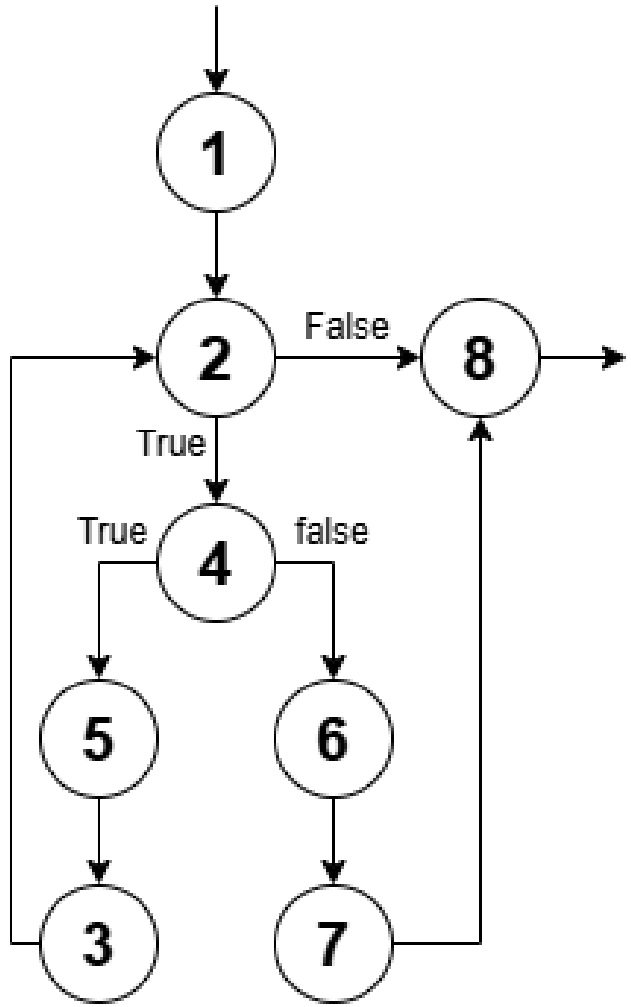
Combined Example

```
for i in range(0, 5, 1): {1,2,3}
    if (i % 2 == 0): {4}
        print(f"{i} is Even") {5}
    else:
        print(f"{i} is Odd") {6}
```



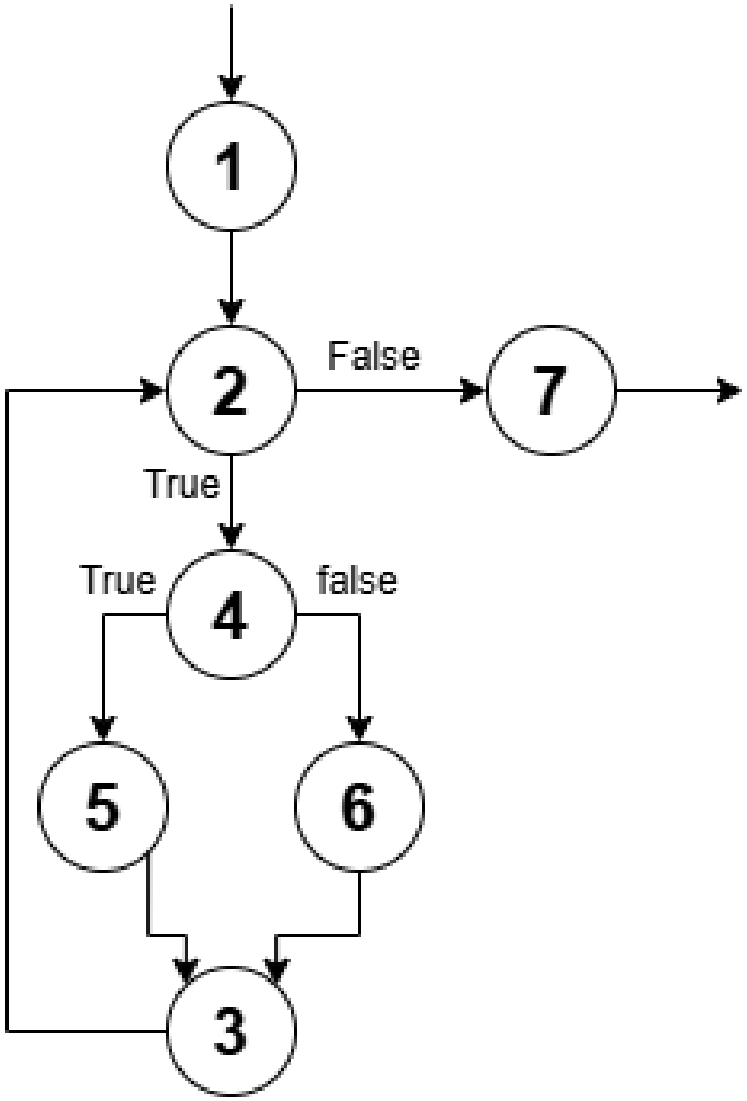
Break

```
for i in range(0, 5, 1): {1,2,3}
  if (i % 2 == 0): {4}
    print(f"{i} is Even") {5}
  else:
    print(f"{i} is Odd") {6}
    break {7}
```



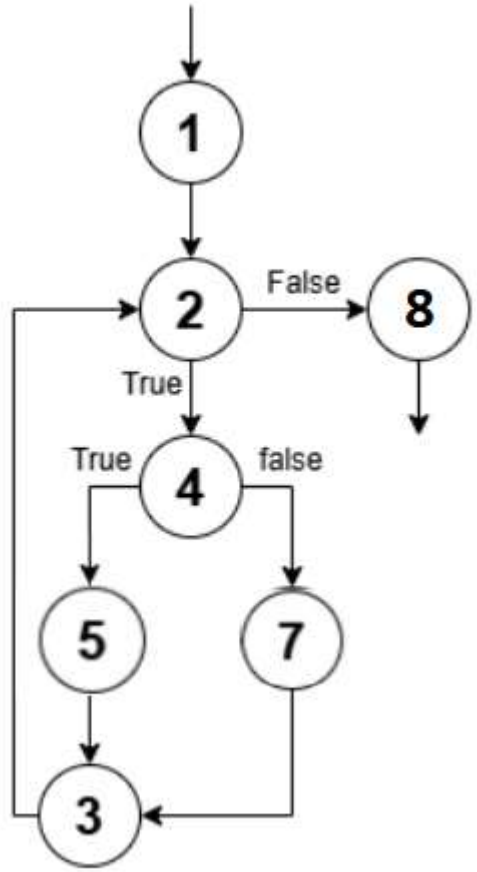
Pass

```
for i in range(0, 5, 1): {1,2,3}
    if (i % 2 == 0): {4}
        pass {5}
        print("test") {5}
    else:
        print(f"{i} is Odd") {6}
```



Continue

```
for i in range(0, 5, 1): {1,2,3}
    if (i % 2 == 0): {4}
        continue {5}
        print("test") {6}
    else:
        print(f"{i} is Odd") {7}
```



Number of Paths

Given a program, how do we exercise all statements and branches at least once?

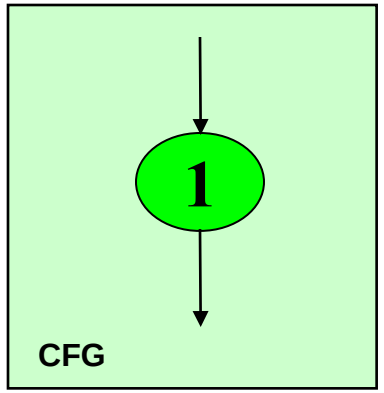
Translating the program into a CFG, an equivalent question is:

- Given a CFG, how do we cover all arcs and nodes at least once?

Since a path is a trail of nodes linked by arcs, this is similar to ask:

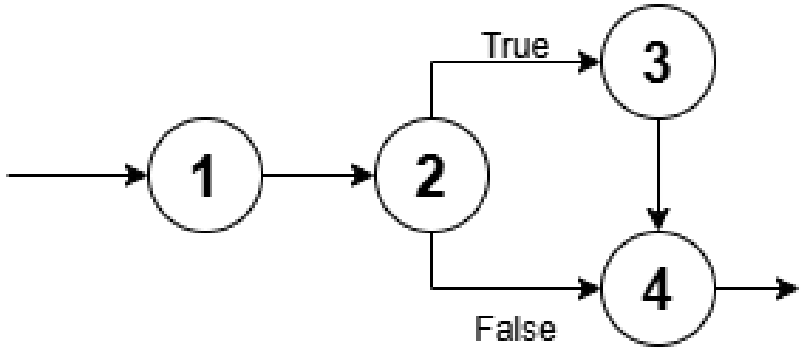
- Given a CFG, what is the set of paths that can cover all arcs and nodes?

Example

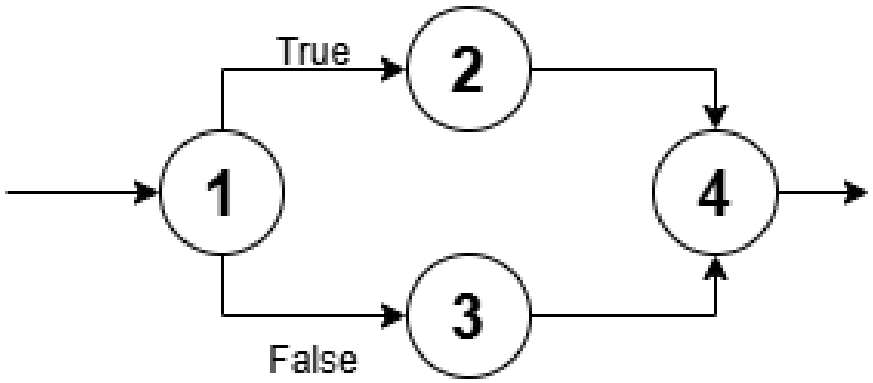


Only **one** path is needed:

- ❑ **[1]**



- **Two** paths are needed:
  - ❑ **[1 – 2 – 4]**
  - ❑ **[1 – 2 – 3 – 4]**



- **Two** paths are needed:
  - ❑ **[1 – 2 – 4]**
  - ❑ **[1 – 3 – 4]**

White Box Testing: Path Based

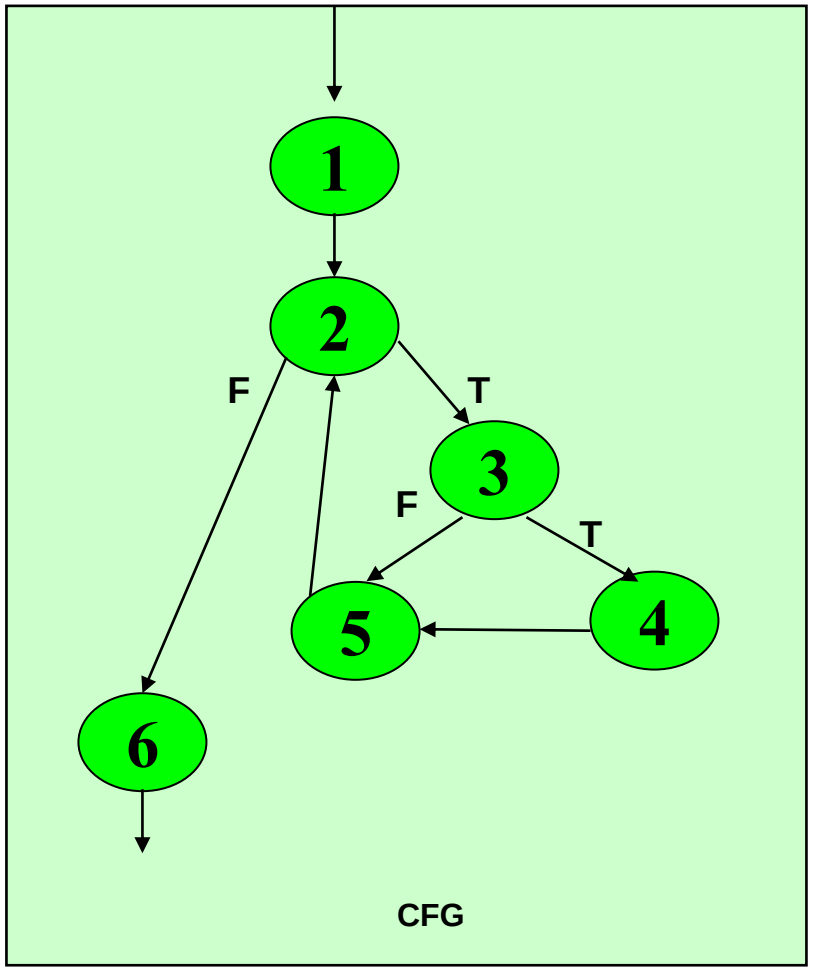
A generalized technique to find out the number of paths needed (known as cyclomatic complexity) to cover all arcs and nodes in CFG.

Steps:

- Draw the CFG for the code fragment.
- Compute the cyclomatic complexity number C, for the CFG.
- Find at most C paths that cover the nodes and arcs in a CFG, also known as Basic Paths Set;
- Design test cases to force execution along paths in the Basic Paths Set.

Path Based Testing: Step 1

```
A = [1, 3, 5 ,8, 9, 43] {1}
min = A[0] {1}
i = 1 {1}
while (i < len(A)): {2}
    if (A[i] < min): {3}
        min = A[i] {4}
    i = i + 1 {5}
print(min) {6}
```



Path Base Testing: Step 2

1. The complexity M is then defined as

$$M = R + 1,$$

where R = the number of regions in the graph.

2. The complexity M is then defined as

$$M = P + 1,$$

where P = the number of predicate nodes in the graph.

3. The complexity M is then defined as

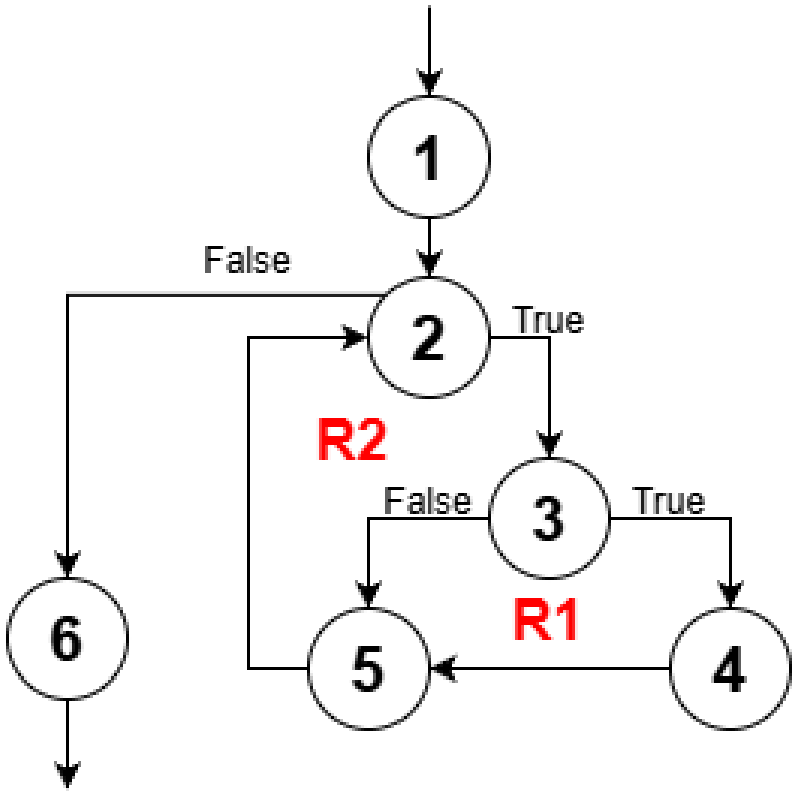
$$M = E - N + 2P,$$ where

- E = the number of edges of the graph.
- N = the number of nodes of the graph.
- P = the number of connected components.

Path Base Testing: Step 2

Cyclomatic complexity M =

- The number of ‘regions’ in the graph(R) + 1
- M = R + 1
- M = 2 + 1 = 3

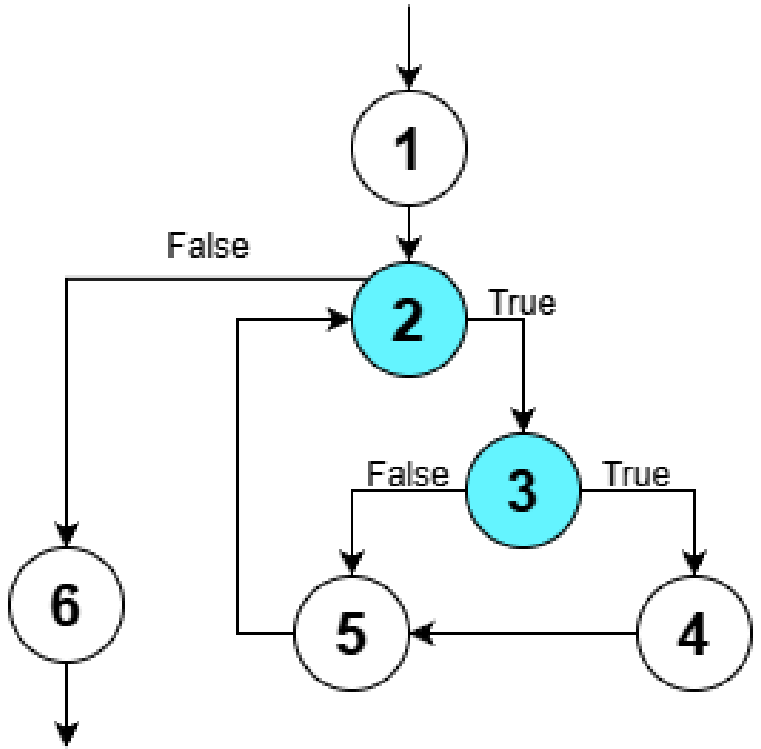


Path Base Testing: Step 2

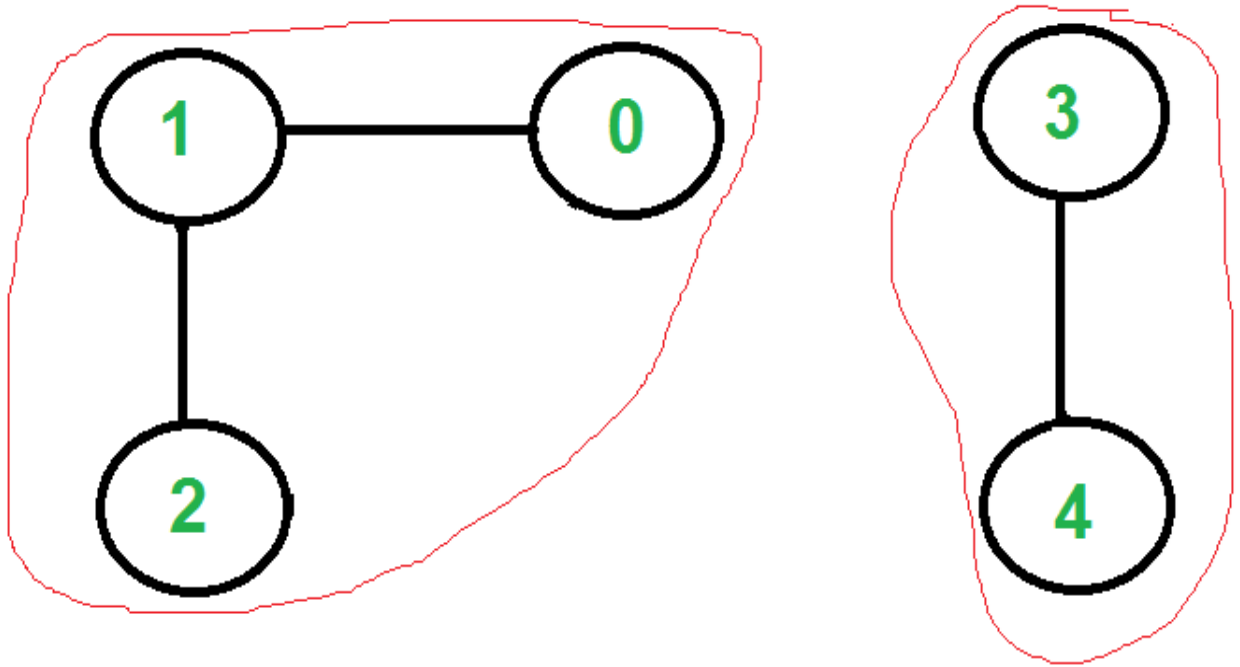
Cyclomatic complexity M =

- The number of ‘predicate’ nodes (P) + 1
- M = P + 1
- M = 2 + 1 = 3

Here, Predicate node means Decision Node



Connected Component in a Graph

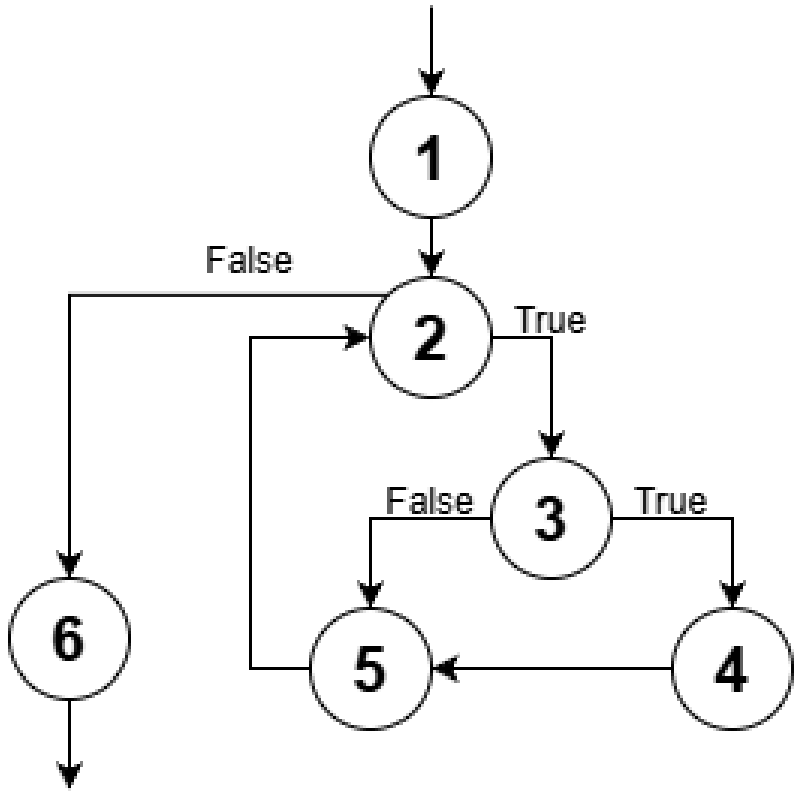


There are two connected components in above undirected graph
0 1 2
3 4

Path Base Testing: Step 2

Cyclomatic complexity, M =
E – N + 2P, Where

- E, edges = 7 (exclude: entry, exit arc)
- N, nodes = 6
- P, connected components = 1
- = 7 – 6 + (2x1)
- = 3



Path Base Testing: Step 3

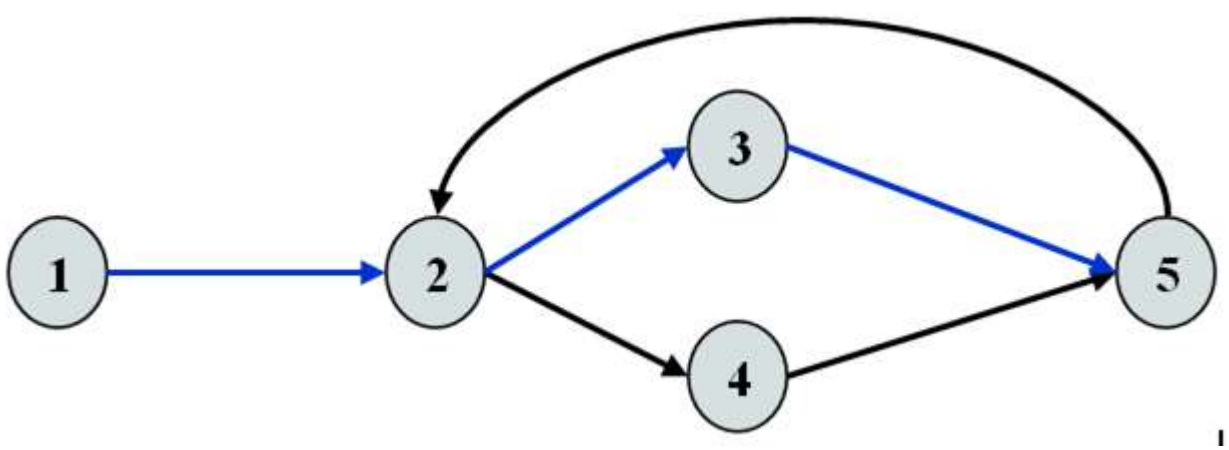
Independent path:

- An **executable** or realizable path through the graph from the start node to the end node that has not been traversed before.
- Must move along **at least one arc that has not been yet traversed** (an unvisited arc).
- The objective is to cover all statements in a program by independent paths.

The number of independent paths to discover <= Cyclomatic complexity number, M

The set of Independent paths is called **Basic Path Set**

Example



M = Regions + 1 = 2 + 1 = 3

1-2-3-5 can be the first independent path; 1-2-4-5 is another; 1-2-3-5-2-4-5 is one more.

Alternatively, if we had identified 1-2-3-5-2-4-5 as the first independent path, there would be no more independent paths.

The number of independent paths therefore can vary according to the order we identify them.

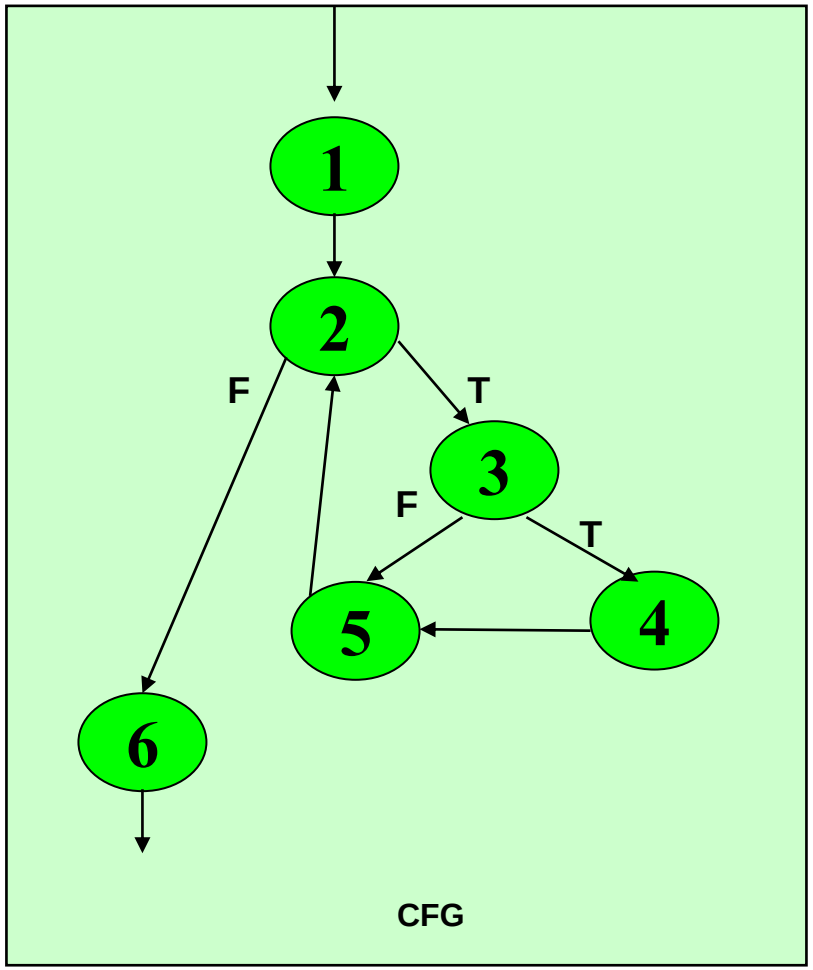
Path Base Testing: Step 3

Cyclomatic complexity = 3.

Need at most 3 independent paths to cover the CFG.

In this example:

- [1 – 2 – 6]
- [1 – 2 – 3 – 5 – 2 – 6]
- [1 – 2 – 3 – 4 – 5 – 2 – 6]



Path Based Testing Example

Prepare a test case for each independent path.

In this example:

Path: [1 – 2 – 6]

Test Case: A = { 5, …}, N = 1
Expected Output: 5

```
A = [5, 3, 5 ,8, 9, 43] {1}
min = A[0] {1}
i = 1 {1}
while (i < N): {2}
    if (A[i] < min): {3}
        min = A[i] {4}
    i = i + 1 {5}
print(min) {6}
```